

Evidências de prompts — G4 AI Master Challenge 002

Hugo Cunha × Claude Code (Fable 5.1)

Gerado em 16/09/2026 21:13 a partir de: prompts.md, PROCESS_LOG.md

Registro de prompts — Hugo Cunha x Claude Code

(Fable 5.1)

Desafio: G4 AI Master Challenge — 002 Redesign de Suporte

Formato: cada entrada = data/hora, prompt literal do Hugo, resumo do que a IA fez em resposta.

P01 — 2026-09-16 17:08 (BRT)

Prompt do Hugo (literal):

Estou participando de um processo seletivo onde a vaga é para IA Master, uma empresa chamada G4. É, temos um desafio que vou precisar da sua ajuda e este desafio precisa ser concluído 100% com IA. Para este desafio, eu tenho que evidenciar todos os comandos que estou dando para IA. Então, no final do desafio, monta um documento em PDF com as evidências dos meus prompts que pedi para você. Nós precisamos de um diferencial, precisamos pensar com a cabeça de um arquiteto de software junto com um gerente de operações. Nós somos um time. Vou te enviar o GitHub para você avaliar o desafio que eu escolhi e, na sequência, vamos iniciar a solução.

<https://github.com/Gestao-Quatro-Ponto-Zero/ai-master-challenge>

Eu escolhi o desafio 002

O que a IA fez:

- Clonou o repositório em `~/Desktop/G4/ai-master-challenge` e leu por inteiro: README raiz, índice de challenges, README do 002, submission-guide, template de submissão e CONTRIBUTING.
- Abriu este registro de evidências (`~/Desktop/G4/evidencias/prompts.md`) a partir do primeiro prompt.
- Verificou o ambiente local (Python, Kaggle CLI, gh) antes de propor o plano.

Achados desta etapa (verificados por código, não por leitura):

- Kaggle liberou download anônimo; os dois datasets oficiais foram baixados e conferidos por SHA-256.
- Dataset 1 tem 8.469 tickets (o brief diz ~30 mil) e é sintético: placeholder `{product_purchased}` em 100% das descrições, resoluções aleatórias, tempos são timestamps numa janela de 27 h (49% "resolvidos" antes da 1ª resposta), satisfação uniforme e sem associação com nenhuma variável ($p > 0,14$). Detalhes em `notas/fatos-datasets.md`.
- Dataset 2 é real (47.837 tickets, 8 classes). Classificador-base TF-IDF+SVM: 86,7% (5-fold). Com gate de confiança $\geq 0,90$: 54% dos tickets roteados com 98,2% de acerto.

- Ferramentas: Claude Code (Fable 5.1) com painel de 5 lentes + 3 críticos adversariais + síntese (workflow multiagente) para avaliar o desafio antes de propor o plano.

P02 — 2026-09-16 ~17:35 (BRT) — proposta de solução do Hugo (transcrição de voz, literal)

Prompt do Hugo (literal):

A minha proposta é criar um sistema de monitoramento de ticket que vai classificar o ticket e identificar se Já ocorreu um problema parecido na nossa, base de dados, ou seja, olhar o passado. e ver como foi resolvido. Basicamente, é um robô de monitoramento, onde ele identifica o ticket aberto, cruza os dados com a nossa base, verifica se já houve tratativa para aquele problema e como foi tratado. E já responde de imediato com aquela tratativa. Se tiver um nível de similaridade acima de 90%. Caso não tenha, O robô vai delegar o ticket para o N2, que é onde entra o atendimento humano. Este atendimento humano é o time técnico que analisa o ticket com os dados já pré-filtrados pela IA. Para isso, nós vamos criar um painel de monitoramento, onde nós vamos medir qual o percentual de tickets atendido pela IA, qual o percentual de tickets delegado para o suporte humano. E nós vamos também classificar o TMA, que é o tempo médio de atendimento de cada nível, de trabalho. Por exemplo, a IA vai fazer o N1, e o time de atendimento humano vai fazer o N2. Neste N2, nós teremos analistas que conseguem identificar o problema sem olhar código, sem acionar outras áreas e responder de bate-pronto, de forma rápida. Caso esse analista não consiga resolver o problema sozinho, sem falar com outras áreas, ele direciona para o time de N3. Que é um analista especialista em chamados envolver outras áreas, envolver tecnicamente aquele problema. Então, é, temos que medir essas três filas de atendimento. E a minha sugestão é usar uma ferramenta Kanban, pode ser o Jira, ou o Trello, ou podemos até mesmo criar uma, que seja institucional. Onde a gente acompanha essas filas e o responsável por cada chamado em tempo real. Então, vamos montar um, protótipo navegável com a identidade da empresa G4, com essas filas de monitoramento que vai gerar também indicadores de medições em tempo real. dos tickets. Você pode montar esse protótipo e subir na minha VPS da Hostinger, só para eu conseguir enviar para eles a acessarem via internet e olharem esse protótipo. Pode ser em PHP, com front-end em React, cem por cento responsivo. É... Não precisa botar autenticação, porque vai ser apenas um protótipo com dados fakes. para a gente navegar e identificar os problemas. Mas basicamente a proposta é ter esse, essa IA que vai fazer essa esse cruzamento de dados e identificar se o problema já foi resolvido anteriormente. E a Arquiteturalmente, nós temos duas saídas. Usar via API com créditos ou fazer um treinamento de uma IA e instalar ela num servidor próprio da empresa carregar a base de dados para a IA utilizar RAG e responder os chamados. [...trecho repetido na transcrição...] Neste caso, mas neste caso eu indico usar por API e também faz uma estimativa de custo dessa utilização.

O que a IA fez: cruzou a proposta com os fatos medidos dos datasets e com os critérios do G4 (protótipo "com dados reais, não cherry-picked"; "automatizar 100% é red flag"); rodou um painel adversarial sobre a proposta antes de responder.

Resultado desta etapa (painéis multiagente, 16 agentes, ~1,65 M tokens):

- Painel 1 (5 lentes + 3 críticos + síntese) avaliou o desafio: ele testa julgamento, não modelagem; a armadilha central é o Dataset 1 sintético; o baseline de IA provavelmente reporta "tempo médio de resolução" e "gargalos" que são ruído. Opção recomendada: triagem com gate por precisão-alvo + auditoria enxuta do Dataset 1 + protótipo rodando sobre TODO o hold-out.
- Painel 2 (4 lentes + 2 críticos + síntese) cruzou a proposta do Hugo com os fatos: manteve N1-IA → N2 → N3, filas medidas, VPS e recomendação de API; corrigiu (a) "responde com a tratativa anterior se similaridade $\geq 90\%$ " — não existe resolução real nos datasets e cosseno $\geq 0,90$ cobre só ~6% dos tickets (1-NN acerta 70,7% vs 86,7% do classificador); (b) "dados fake" — cards devem ser tickets reais do hold-out; (c) TMA por nível — imensurável nos datasets, entra rotulado como simulação; (d) PHP + React — o modelo é Python; recomendação de um só runtime. Estimativa de custo de API com preços lidos em 16/09/2026.
- Riscos práticos confirmados por código: `.gitignore` do repo do G4 ignora `submissions/` (PR sairia vazio sem `git add -f`); `gh` está logado na conta StartIACode, não na pessoal do Hugo.
- Síntese entregue ao Hugo com as decisões que só ele pode tomar. Nenhuma implementação iniciada antes da aprovação (roteiro de brainstorming: gate de aprovação).

P03 — 2026-09-16 ~18:40 (BRT) — decisões do Hugo + nova proposta (template de fechamento)

Prompt do Hugo (literal):

Sobre a similaridade, não temos dados reais nem a base de dados deles, entendo que no desafio eles colocaram dados descaracterizados. Então vamos usar sim, mas cria uma tela de exemplo de como ficaria após padronização. E aí que entra minha nova proposta que fica um passo antes: criar um template de fechamento de ticket. Quando o ticket é fechado hoje manualmente pelo time, temos que obrigar a preencher os dados no padrão para alimentarmos a base para o RAG da IA. Ou seja, pegar a base atual de produção e reclassificar o fechamento, pois aí sim teremos similaridade acima de 90%

Então cria um formulário de exemplo, só pra ilustrar mesmo. Porque na verdade eles vão fechar na ferramenta atual que eles usam (não sei qual)

1 pode usar minha conta da StartIA 2 ok 3 ok 4 seja mais claro 5 ok desligado 6 ok

Decisões tomadas pelo Hugo: conta GitHub StartIACode para fork/PR; stack Python FastAPI + página única; deploy na VPS como último passo; LLM desligado na demo; tom sobre o Dataset 1 em um parágrafo com tabela. **Pedido de esclarecimento:** o que significa o limiar padrão do gate (item 4). **Adição do Hugo (julgamento humano que a IA não trouxe sozinha):** o template de fechamento obrigatório como passo anterior à similaridade: padronizar o fechamento na ferramenta

atual e retro-padronizar a base de produção para alimentar a base de conhecimento (RAG). A similaridade continua no desenho, com uma tela de exemplo "como ficaria após a padronização", rotulada como ilustrativa.

O que a IA fez: explicou o limiar; escreveu a especificação do desenho aprovado; preparou fork/branch; iniciou a construção.

Execução após o P03 (registro do que a IA fez, com os tropeços):

- Escreveu a especificação aprovada em `submissions/hugo-cunha/docs/2026-09-16-desenho-solucao.md` (inclui a explicação do limiar pedida no item 4) e o plano em `docs/2026-09-16-plano-implementacao.md` (13 tarefas com contratos de API fixos).
- Fork criado em `StartIACode/ai-master-challenge`; branch `submission/hugo-cunha`; identidade git local "Hugo Cunha startia.develop@gmail.com".
- Erro da IA, corrigido: o primeiro `git add -f` (necessário porque o `.gitignore` do G4 ignora `submissions/`) arrastou o `.venv/` com 8.841 arquivos e os CSVs extraídos. Correção: `git rm --cached, script tools/git-add-safe.sh` com exclusões explícitas e commit corrigido (15 arquivos rastreados).
- Workflow multiagente de construção iniciado (agentes por tarefa + revisores com correção), sem commits pelos agentes.

P04 — 2026-09-16 ~21:00 (BRT)

Prompt do Hugo (literal):

me passa um status da construçao

O que a IA fez: respondeu com o status por etapa (concluído / em andamento / pendente), pediu o LinkedIn para o template e reexplicou o limiar do gate em linguagem de gestor.

Fechamento da construção (após o P04):

- Interface aprovada em segunda revisão; 4 observações leves aplicadas; 105 testes verdes.
- Prova de setup em clone limpo do fork: `git clone + make setup + make train + make test` em 2 min 00 s.
- Deploy na VPS: primeira tentativa falhou (Python do `uv` em `/root`, inacessível ao `www-data`); script corrigido; segunda tentativa no ar em `https://g4-triagem.187-77-249-237.sslip.io` com Let's Encrypt e vhost padrão intacto. Replay testado no site público pelo navegador, sem erros de console.
- README no template do G4, process log, prompts literais e `evidencias.pdf` comitados e publicados no fork (branch `submission/hugo-cunha`, 71 arquivos, todos dentro de `submissions/hugo-cunha/`).
- PR **não** aberto ainda: aguarda o LinkedIn e o OK do Hugo.

Process log — como a IA foi usada (e onde errou)

Candidato: Hugo Cunha · Ferramenta: Claude Code (modelo Claude Fable 5.1) no desktop, sessão única iniciada em 16/09/2026 às 17:08 (BRT). Prompts literais em [prompts.md](#); PDF em [evidencias.pdf](#); histórico git no branch `submission/hugo-cunha`.

1. Regra do trabalho

Tudo foi feito com IA, inclusive este texto. O meu papel foi decidir: o que pedir, quando desconfiar, o que cortar e o que só quem opera sabe. Cada decisão minha está marcada como **[Hugo]** abaixo; cada erro da IA que precisou de correção está em §4.

2. Linha do tempo (horas reais)

Hora (BRT)	Etapa	Prompt	O que aconteceu
17:08–17:30	Entender o desafio e os dados	P01	Clone do repositório, leitura integral do brief, download dos dois datasets direto do Kaggle com SHA-256, profilagem por código. Achado: Dataset 1 tem 8.469 linhas (não ~30 mil) e é sintético.
17:30–18:10	Avaliar o desafio por várias cabeças	(IA)	Painel multiagente: 5 lentes (avaliador do G4, gerente de operações, arquiteto, cientista cético, comunicação) + 3 críticos hostis + síntese.
17:35	Minha proposta de solução	P02	[Hugo] Robô N1-IA que cruza com o histórico e responde se similaridade $\geq 90\%$; N2/N3 humanos; painel com % IA vs humano e TMA por nível; protótipo na minha VPS; API $\times$ modelo próprio com custo.
17:50–18:10	Cruzar minha proposta com os fatos	(IA)	Segundo painel (4 lentes + 2 críticos): manteve N1/N2/N3, filas, VPS e API; corrigiu similaridade, "dados fake", TMA e stack. Preços de API lidos na hora.
18:10–18:40	Avaliação entregue + decisões	P03	[Hugo] Conta GitHub, stack Python, VPS por último, LLM desligado, tom sobre o Dataset 1. [Hugo] Nova ideia: template de fechamento obrigatório + retro-padronização da base como passo anterior à similaridade.
18:45–19:20	Especificação, plano, fork, esqueleto	(IA)	Spec e plano com contratos de API fixos; fork; primeiro commit (com um tropeço no <code>.gitignore</code> , §4).
19:20–19:45	Construção, onda 1	(IA)	Agentes em paralelo: normalização + política + rascunhos (34 testes), diagnóstico do Dataset 1, treino do modelo (86,5% no hold-out, artefatos e figuras).
19:45–20:10	Construção, onda 2	(IA)	Backend (modelo em runtime, replay, board SQLite, template de fechamento, API; 104 testes) + revisor independente; documentos de diagnóstico e proposta escritos a partir dos JSONs.
20:10–21:05	Construção, onda 3	(IA)	Página única (7 abas), verificação no navegador com 16 screenshots, revisor achou 1 problema alto (reset do formulário), corrigido e reaprovado; correções leves do revisor do backend aplicadas.

Hora (BRT)	Etapa	Prompt	O que aconteceu
21:00	Pedido de status	P04	[Hugo] Cobrou status; recebeu o quadro do que estava feito, em andamento e pendente.
21:05–21:40	Prova de setup, deploy, documentação	(IA)	Clone limpo do fork cronometrado (2 min 00 s até 105 testes verdes); deploy na VPS falhou na primeira tentativa (Python do uv dentro de <code>/root</code> , inacessível ao <code>www-data</code>), script corrigido e reexecutado; README no template; este log; PDF de evidências.
depois	PR	(IA)	Abertura do PR após o OK do Hugo.

3. Como decompus o problema antes de promptar

- Auditar antes de analisar.** Baixar os arquivos reais e medir (linhas, hashes, distribuições, testes de associação) antes de qualquer proposta. Foi isso que mudou tudo: o brief pedia gargalos "com dados" e o Dataset 1 não sustenta nenhum.
- Lentes, não um prompt.** Em vez de pedir "resolva o desafio", pedi que o desafio fosse avaliado por papéis distintos e depois atacado por críticos. A síntese virou a avaliação que eu li e decidi.
- Minha proposta como hipótese.** A visão N1-IA → N2 → N3 é minha; a IA testou cada parte contra os dados e devolveu o que sobrevive e o que muda.
- Contratos antes de código.** Especificação e plano com contratos de API fixos, para que agentes construíssem partes em paralelo e um revisor independente checasse cada etapa.
- Um só lugar para os números.** Todo número dos documentos vem de `metrics.json` e `ds1_metrics.json`, gerados por `make train`.

4. Onde a IA errou e como corrigi (com o par prompt → saída)

#	O que a IA fez	Como foi pego	Correção
1	Aceitaria o brief ("~30.000 registros com texto real") sem conferir.	Profilagem por código logo no P01.	8.469 linhas, placeholder em 100% das descrições, tempos numa janela de 27 h: o Dataset 1 virou controle negativo, não fonte de gargalos.
2	Minha proposta P02 dizia "responde com a tratativa anterior se similaridade ≥ 90%". A IA poderia ter simplesmente construído isso.	Painel 2 mediu: cosseno ≥ 0,90 cobre 5,1% dos tickets e o vizinho acerta 70,2% contra 86,5% do classificador; não há campo de resolução em nenhum dataset.	A decisão passou a ser a confiança do classificador; a similaridade virou evidência ao N2. [Hugo] Respondi com o template de fechamento obrigatório, que cria a base que faltava.
3	O primeiro <code>git add -f</code> (necessário porque o <code>.gitignore</code> do G4 ignora <code>submissions/</code>) arrastou o <code>.venv/</code> com 8.841 arquivos e os CSVs.	Contagem de arquivos rastreados no commit.	<code>git rm --cached</code> , script <code>tools/git-add-safe.sh</code> com exclusões explícitas, commit corrigido (15 arquivos).

#	O que a IA fez	Como foi pego	Correção
4	O plano de implementação trazia uma lista de stopwords com "hello" e, ao mesmo tempo, um teste que exigia "hello" no resultado.	O agente da tarefa rodou o teste e ele falhou.	O agente tratou o teste como contrato e documentou a inconsistência do plano.
5	Primeira versão da busca de similares usava blocos de 2.000 linhas e chegou a 3,3 GB de pico de RAM num Mac de 8 GB.	Medição de RSS durante o treino.	Blocos de 125 linhas; pico 1,3 GB; documentado no script.
6	Figura do Dataset 1 saiu com anotações sobrepostas e ponto decimal em vez de vírgula.	O agente abriu o PNG antes de entregar.	Eixo com folga, subtítulo separado, formatação pt-BR.
7	Os 3 exemplos fixos da demo (seed 7) caíam em um Administrative rights com $p = 0,54$ e dois Miscellaneous: nenhum caso de auto-rotação para mostrar.	Revisor independente do backend.	Seleção determinística de um trio representativo (auto-rotação, sugerir, baixa confiança).
8	Motivos exibidos ao analista com ponto decimal e "1.00" por arredondamento.	Revisor independente.	Vírgula decimal e "0,99+" acima de 0,995; teste atualizado.
9	Os documentos citaram 65,3% e 90,8% onde o exato era 65,2% e 90,7%.	Verificador de formatação do próprio agente de documentos.	Corrigido para os valores exatos dos JSONs.
10	O botão "Preencher exemplo válido" e o fluxo "Resolver → Registrar fechamento" chamavam <code>form.reset()</code> e depois preenchiam os campos; o listener de reset limpava o que tinha acabado de ser preenchido (subcategorias dependentes vazias).	Revisor independente da interface reproduziu no navegador.	Reset programático sem disparar o listener; screenshot refeito; segunda revisão aprovada.
11	Na verificação da interface, o agente navegou para outra aba por <code>#fragmento</code> achando que era recarga de página: o CSS editado não foi recarregado e ele quase deu por corrigido um erro que continuava.	O console ainda mostrava o 404 antigo.	Recarga completa forçada e nova verificação em sessão limpa.
12	A tabela de política em 375 px saiu com linhas de até 161 px de altura porque uma regra de largura mínima espremia a coluna Motivo.	Screenshot em mobile.	Regra removida, largura mínima só na última coluna, screenshot refeito.
13	A política dizia que Storage tinha "a melhor precisão entre as classes"; o JSON mostra Purchase acima (0,96 contra 0,94).	Agente dos documentos cruzou o texto com <code>per_class</code> .	Motivo corrigido para "entre as classes auto-rotação"; <code>policy.json</code> e a proposta regenerados.
14	O deploy na VPS instalou o Python gerenciado pelo <code>uv</code> em <code>/root/.local</code> , e o serviço roda como <code>www-data: Permission denied</code> em loop de reinício. O script seguiu como se	<code>systemctl status</code> e <code>journalctl</code> na VPS.	Python instalado em <code>/opt/uv-python</code> , <code>venv</code> recriado, espera ativa de até 90 s pelo <code>/api/health</code> ; deploy refeito.

#	O que a IA fez	Como foi pego	Correção
	tivesse dado certo porque a verificação era um <code>sleep 3</code> e um <code>curl</code> .		

5. O que eu adicionei que a IA sozinha não faria [Hugo]

- **A arquitetura em três níveis com a IA na frente e humanos atrás** (N1-IA → N2 → N3), com o N2 definido por "resolve sem olhar código nem acionar outras áreas" e o N3 por "envolve outras áreas". A IA transformou isso em gate e política por classe, mas o desenho operacional é meu.
- **O template de fechamento obrigatório e a retro-padronização da base.** Quando os dados mostraram que não existe "como foi resolvido", a resposta não foi desistir da similaridade, foi criar o processo que gera a base: fechar o ticket na ferramenta atual só com os campos padronizados, e reclassificar o histórico com IA para alimentar a base.
- **Recomendar API em vez de modelo próprio** e pedir a conta: dezenas de dólares por mês contra R\$ 13 a 17 mil por ano de GPU própria.
- **As decisões de escopo:** um só runtime Python (em vez de PHP + React), LLM desligado na demo, VPS como último passo, tom de um parágrafo sobre o Dataset 1 em vez de manchete.
- **A postura de time:** arquiteto de software e gerente de operações na mesma mesa. O arquiteto exigiu contratos, testes e reprodutibilidade; o gerente exigiu que a IA nunca responda ao cliente nem feche ticket, e que o painel mostre erros na tela.

6. Iterações

- Prompts meus: P01 a P04 (avaliar, propor, decidir e cobrar status) e 3 workflows multiagente: avaliação (9 agentes), reconciliação da minha proposta (7 agentes) e construção com revisores (10 agentes, 2 rodadas de revisão na interface).
- Commits no branch, um por etapa fechada e testada (34 → 104 → 105 testes); histórico completo em `git log`.
- Revisão independente do backend (aprovada, 5 observações leves aplicadas) e da interface (1 problema alto corrigido, segunda revisão aprovada com 4 observações leves aplicadas).